

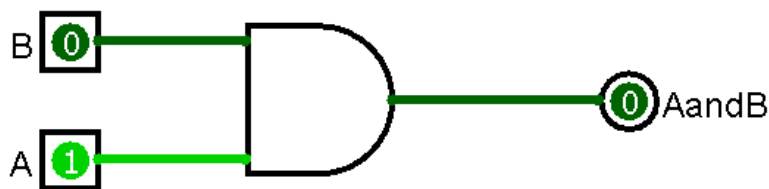
I will explain the following:

1. And gate implementation
2. And gate with a single input negated implementation
3. 1-bit comparator implementation (where each number has a single bit)
4. 2-bit comparator implementation
5. 4-bit comparator implementation

1. And Gate implementation

The `and` gate is a basic and simple gate, therefore it can be easily described using a “Data flux” architecture.

Representative drawing (Made using Logisim, obviously):



Truth table:

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

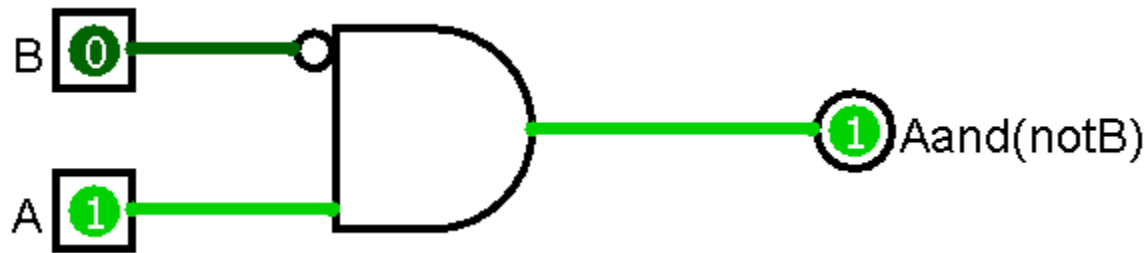
VHDL code:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity and2 is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          AandB : out STD_LOGIC);
end and2;
architecture Behavioral of and2 is
begin
    AandB <= A and B;
end Behavioral;
```

It's self-explanatory – I don't think I need to explain how I came up with this.

2. And Gate with a single input negated implementation

Representative Drawing:



Truth table:

A	B	A and (not B)
0	0	0
0	1	0
1	0	1
1	1	0

VHDL Implementation:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity AandNotB is
    Port ( in1 : in STD_LOGIC;
          in2 : in STD_LOGIC;
          outp : out STD_LOGIC);
end AandNotB;
architecture Behavioral of AandNotB is
    component and2 is
        Port ( A : in STD_LOGIC;
              B : in STD_LOGIC;
              AandB : out STD_LOGIC);
    end component;
    signal notin2 : std_logic ;
begin
    notin2 <= (not in2);
    c1 : and2 port map (in1,notin2,outp);
end Behavioral;
```

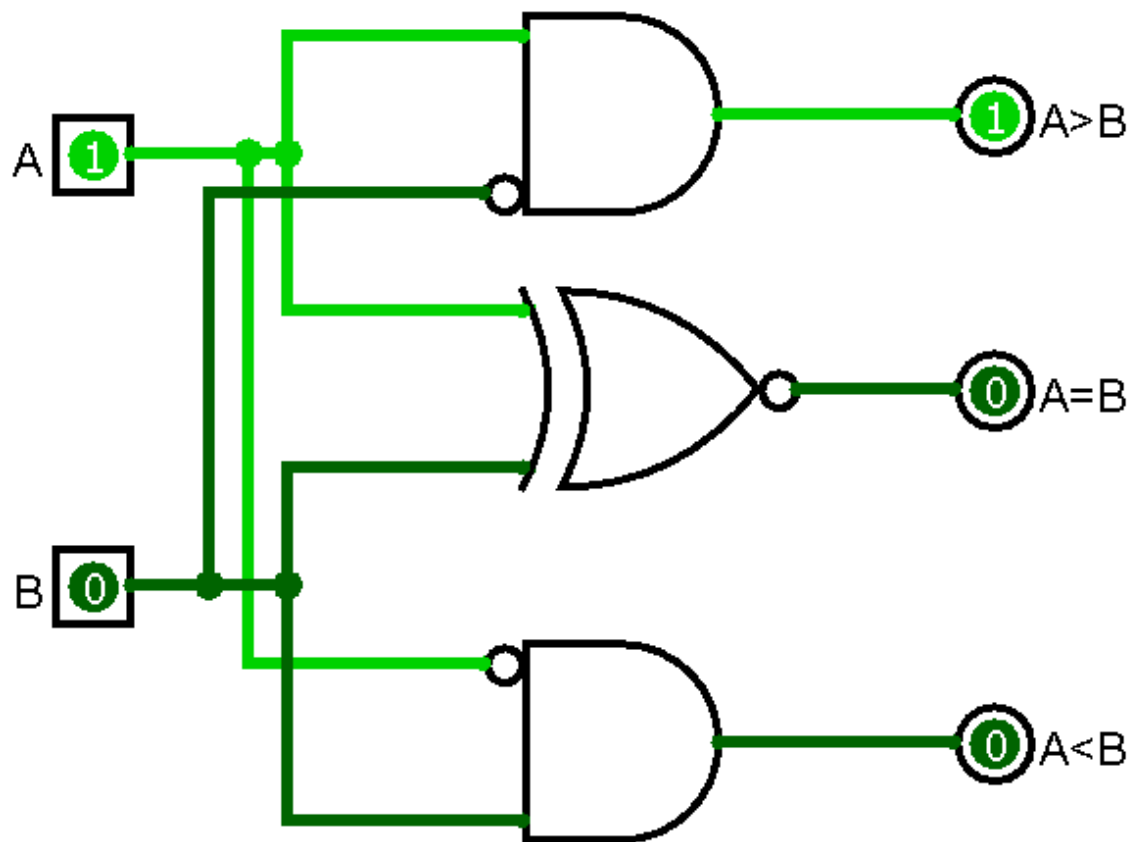
This is the next building block toward the 4-bit comparator. It's a relatively simple gate that outputs a '1' only when bit A is bigger than bit B (as seen in the truth table). Swapping the places of these bits, gives us the situation where A is smaller than B.

One other detail I'd like to mention before going on is the fact that the XNOR gate outputs a '1' only when both bits are equal. Using these 3 building blocks we can arrive at the one-bit comparator.

A	B	A XNOR B
0	0	1
0	1	0
1	0	0
1	1	1

3. 1-bit comparator implementation

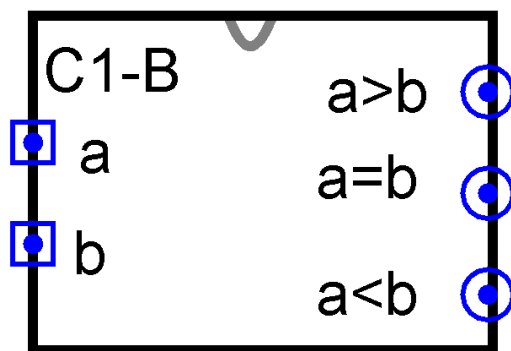
Representative Drawing:



Truth Table :

A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Black Box:



VHDL Implementation:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity oneBitComp is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          AbB : out STD_LOGIC;
          AeB : out STD_LOGIC;
          AsB : out STD_LOGIC);
end oneBitComp;
architecture Behavioral of oneBitComp is
    component AandNotB is
        Port ( in1 : in STD_LOGIC;
              in2 : in STD_LOGIC;
              outp : out STD_LOGIC);
    end component;
begin
    c1 : AandNotB port map ( in1 => A, in2=> B, outp => AbB);
    AeB <= A xnor B;
    c2 : AandNotB port map ( in1 => B, in2=> A, outp => AsB);
end Behavioral;

```

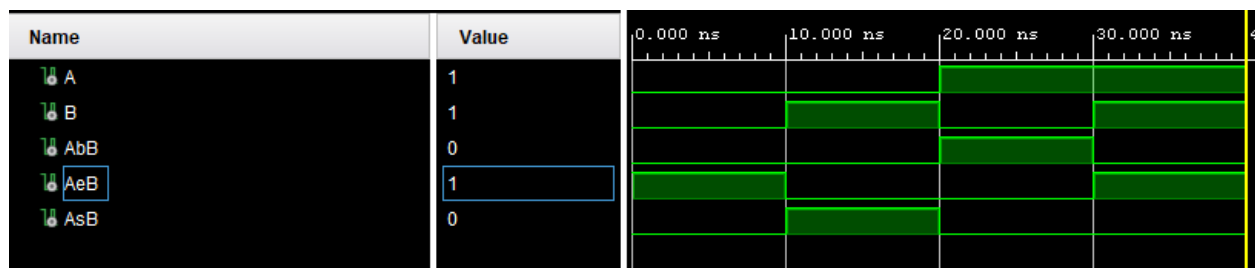
VHDL tests:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity sim_comp_one_bit is
end sim_comp_one_bit;
architecture Behavioral of sim_comp_one_bit is
    component oneBitComp
        Port ( A : in STD_LOGIC;
              B : in STD_LOGIC;
              AbB : out STD_LOGIC;
              AeB : out STD_LOGIC;
              AsB : out STD_LOGIC);
    end component;
    signal A, B : STD_LOGIC;
    signal AbB, AeB, AsB : STD_LOGIC;
begin
    C1: oneBitComp port map (A => A,B => B,
        AbB => AbB, AeB => AeB,AsB => AsB);
    A <= '0', '0' after 10 ns, '1' after 20 ns, '1' after 30 ns;
    B <= '0', '1' after 10 ns, '0' after 20 ns, '1' after 30 ns;
end Behavioral;

```

VHDL Wave-function Simulation:



The simulation works as such:

1. A and B start with value '0'.
2. B takes on value '1' after 10 ns.
3. A becomes '1' and B becomes '0' after 20ns.
4. B becomes '1' after 30 ns.
5. Retains this state moving forward.

It behaves as described in the truth table above. When B is bigger than A, AsB is 1 (s for smaller). When A is bigger than B, AbB is 1 (b for bigger). And when they're equal, AeB is 1, as it should (e stands for equal). I hope this is proof enough – moving on to the next step.

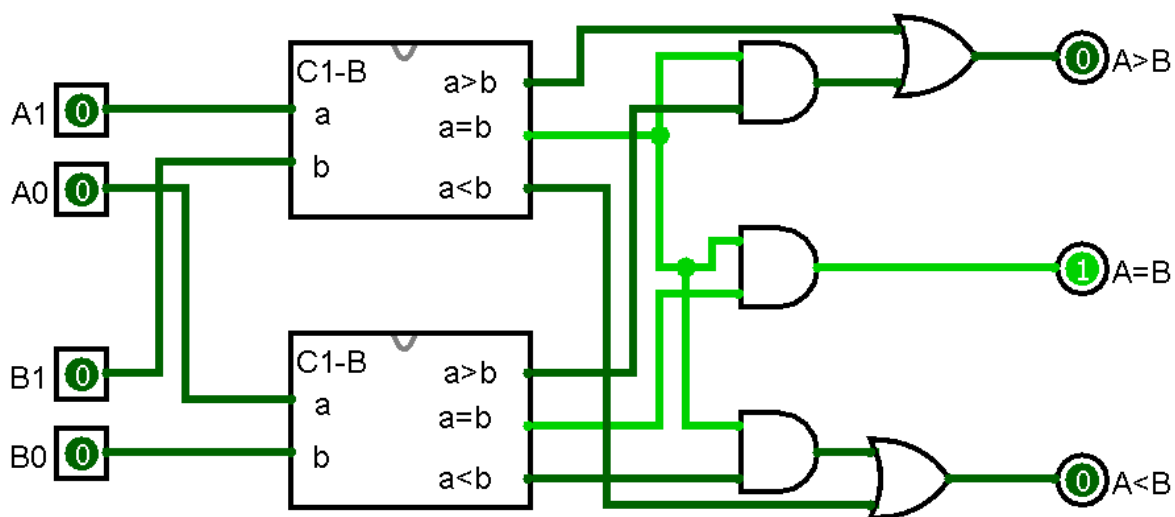
4. 2-bit comparator implementation

Before moving on, I must explain the logic behind cascading bit comparators, let's look at how we're usually comparing 2-bit numbers, say $A=A_1A_0$ and $B=B_1B_0$:

1. $A > B$: if $(A_1 > B_1 \text{ or } (A_1 == B_1 \text{ and } A_0 > B_0))$
2. $A = B$: if $(A_1 == B_1 \text{ and } A_0 == B_0)$
3. $A < B$: if $(A_1 < B_1 \text{ or } (A_1 == B_1 \text{ and } A_0 < B_0))$

As we can observe, the more significant bits have a higher power of 'decision' when compared to the less significant ones. We will apply this logic later when it comes to 4-bit comparisons. For this step however, we will use 1-bit comparators to get the result of each condition, paired with some basic logic gates, found in the expression itself.

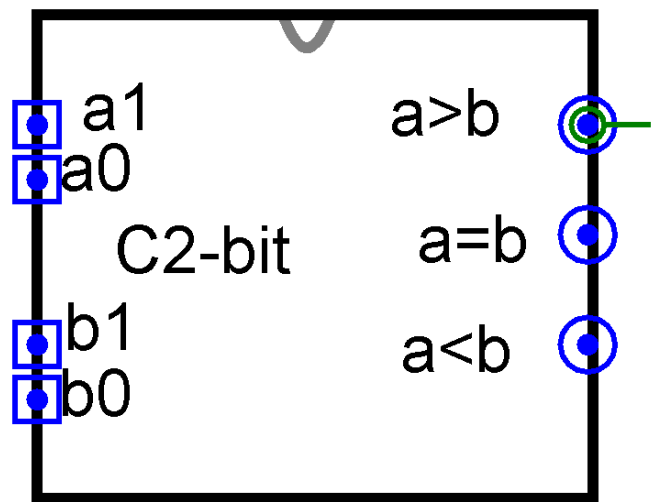
Representative drawing:



Truth table:

A1	A0	B1	B0	A>B	A=B	A<B
0	0	0	0	1	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

Black box:



VHDL implementation:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity twoBitComp is
    Port ( A1 : in STD_LOGIC;
          A0 : in STD_LOGIC;
          B1 : in STD_LOGIC;
          B0 : in STD_LOGIC;
          A1A0_b_B1B0 : out STD_LOGIC;
          A1A0_e_B1B0 : out STD_LOGIC;
          A1A0_s_B1B0 : out STD_LOGIC);
end twoBitComp;
architecture Behavioral of twoBitComp is
    component oneBitComp is
        Port ( A : in STD_LOGIC;
              B : in STD_LOGIC;
              AbB : out STD_LOGIC;
              AeB : out STD_LOGIC;
              AsB : out STD_LOGIC);
    end component;
    component and2 is
        Port ( A : in STD_LOGIC;
              B : in STD_LOGIC;
              AandB : out STD_LOGIC);
    end component;

```

(VHDL implementation continuation):

```
    signal AlbB1, AleB1, AlsB1, A0bB0, A0eB0, A0sB0, andBigger, andSmaller : std_logic ;
begin
C1 : oneBitComp port map ( A => A1, B=> B1, AbB => AlbB1, AeB => AleB1, AsB => AlsB1);
C2 : oneBitComp port map ( A => A0, B=> B0, AbB => A0bB0, AeB => A0eB0, AsB => A0sB0);
C3 : and2 port map ( A => AleB1, B => A0sB0, AandB => andSmaller);
C4 : and2 port map ( A=> AleB1, B=>A0bB0, AandB => andBigger);
AlA0_e_B1B0 <= AleB1 and A0eB0;
AlA0_s_B1B0 <= AlsB1 or andSmaller;
AlA0_b_B1B0 <= AlbB1 or andBigger;
end Behavioral;
```

Each signal inside the black box has in it's name what bits it compares and what results should hold (b – bigger, e – equal , s -smaller), and the 'andBigger', 'andSmaller' signals are used for the cases where we need to check the less significant bits for a conclusion.

VHDL simulation code:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity sim_two_bit_comp is
end sim_two_bit_comp;
architecture Behavioral of sim_two_bit_comp is
    component twoBitComp
        Port ( A1 : in STD_LOGIC;
              A0 : in STD_LOGIC;
              B1 : in STD_LOGIC;
              B0 : in STD_LOGIC;
              AlA0_b_B1B0 : out STD_LOGIC;
              AlA0_e_B1B0 : out STD_LOGIC;
              AlA0_s_B1B0 : out STD_LOGIC);
    end component;
    signal A1, A0, B1, B0 : STD_LOGIC := '0';
    signal AlA0_b_B1B0, AlA0_e_B1B0, AlA0_s_B1B0 : STD_LOGIC;

begin
C1: twoBitComp port map ( A1 => A1,A0 => A0,B1 => B1,B0 => B0,
    AlA0_b_B1B0 => AlA0_b_B1B0,AlA0_e_B1B0 => AlA0_e_B1B0,AlA0_s_B1B0 => AlA0_s_B1B0);
    B0 <= '0', '1' after 10 ns, '0' after 20 ns, '1' after 30 ns,
        '0' after 40 ns, '1' after 50 ns, '0' after 60 ns, '1' after 70 ns,
        '0' after 80 ns, '1' after 90 ns, '0' after 100 ns, '1' after 110 ns,
        '0' after 120 ns, '1' after 130 ns, '0' after 140 ns, '1' after 150 ns;
    B1 <= '0', '1' after 20 ns, '0' after 40 ns, '1' after 60 ns,
        '0' after 80 ns, '1' after 100 ns, '0' after 120 ns, '1' after 140 ns;
    A0 <= '0', '1' after 40 ns, '0' after 80 ns, '1' after 120 ns;
    A1 <= '0', '1' after 80 ns;
end Behavioral;
```

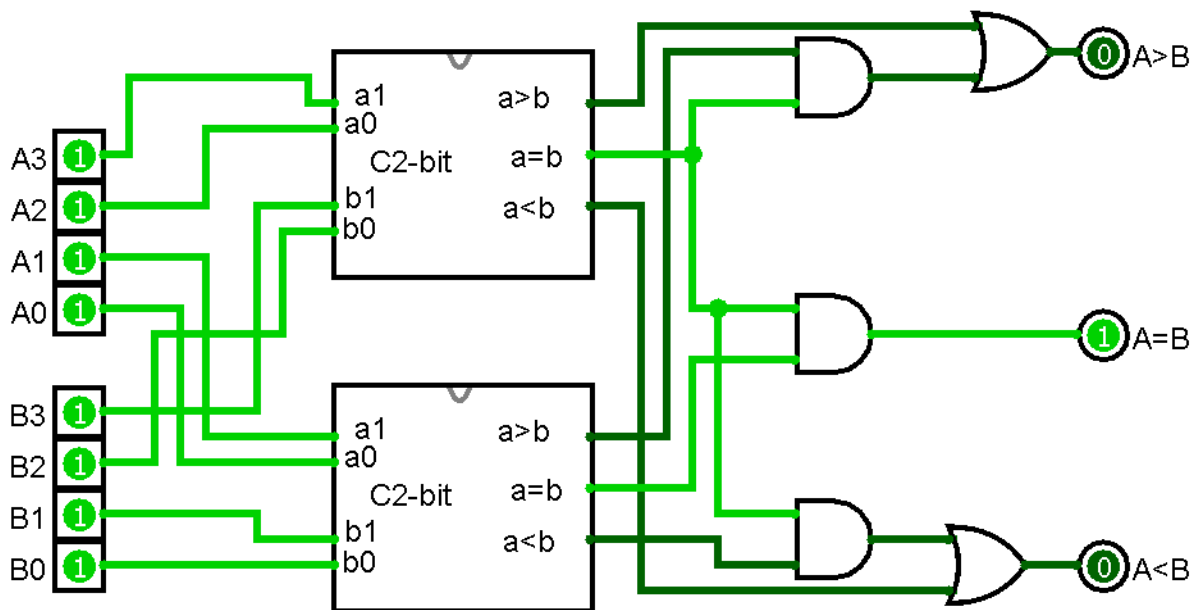
VHDL wave-function simulation results:



5. 4-bit comparator implementation

The thought process behind this, as mentioned before, is checking the more significant bits first, and then the lower ones. If a conclusion based on the upper bits is reached, the lower conclusion can be, as a figure of speech, ignored.

Representative drawing:



VHDL implementation:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Four_bit_comparator is
    Port ( A_0 ,A_1 ,A_2 ,A_3 : in STD_LOGIC;
           B_0 , B_1 , B_2 ,B_3 : in STD_LOGIC;
           AbiggerB , AequalB ,AsmallerB : out STD_LOGIC);
end Four_bit_comparator;
architecture Behavioral of Four_bit_comparator is
    component twoBitComp is
        Port ( A1 ,A0 : in STD_LOGIC;
              B1 ,B0 : in STD_LOGIC;
              A1A0_b_B1B0 , A1A0_e_B1B0, A1A0_s_B1B0 : out STD_LOGIC);
    end component;
    component and2 is
        Port ( A ,B : in STD_LOGIC;
              AandB : out STD_LOGIC);
    end component;
    signal A3A2bB3B2,A3A2eB3B2,A3A2sB3B2,A1A0bB1B0,A1A0eB1B0,A1A0sB1B0, andSmaller, andBigger : std_logic ;

begin
    C1 : twoBitComp port map (A1 => A_3, A0 => A_2, B1 => B_3, B0 => B_2,
                             A1A0_b_B1B0 => A3A2bB3B2, A1A0_e_B1B0 => A3A2eB3B2, A1A0_s_B1B0 => A3A2sB3B2);
    C2 : twoBitComp port map (A1 => A_1, A0 => A_0, B1 => B_1, B0 => B_0,
                             A1A0_b_B1B0 => A1A0bB1B0, A1A0_e_B1B0 => A1A0eB1B0, A1A0_s_B1B0 => A1A0sB1B0);
    C3 : and2 port map ( A => A3A2eB3B2, B=> A1A0sB1B0, AandB => andSmaller);
    C4 : and2 port map ( A => A3A2eB3B2, B=> A1A0bB1B0, AandB => andBigger);
    AbiggerB <= A3A2bB3B2 or andBigger;
    AequalB <= A3A2eB3B2 and A1A0eB1B0;
    AsmallerB<= A3A2sB3B2 or andSmaller;
end Behavioral;
```

Each signal contains a partial conclusion based on the comparison (i.e. A3A2eB3B2 is ‘1’ if the named bits form an equal or not number). “andSmaller” and “andBigger” refer to the case where the more significant half of the bits are equal, and a “deeper” dive is required.

VHDL simulation:

Since writing 511 “‘0’/‘1’ after Xns” isn’t practical, I resorted to toggling each value after a corresponding time-period.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity sim_comp_four_bits is
end sim_comp_four_bits;
architecture Behavioral of sim_comp_four_bits is
    component Four_bit_comparator
        Port (
            A_0, A_1, A_2, A_3 : in STD_LOGIC;
            B_0, B_1, B_2, B_3 : in STD_LOGIC;
            AbiggerB, AequalB, AsmallerB : out STD_LOGIC);
    end component;
    signal A_0, A_1, A_2, A_3 : STD_LOGIC := '0';
    signal B_0, B_1, B_2, B_3 : STD_LOGIC := '0';
    signal AbiggerB, AequalB, AsmallerB : STD_LOGIC;
begin

```

(continuation):

```

begin
C1: Four_bit_comparator port map (
    A_0 => A_0, A_1 => A_1, A_2 => A_2, A_3 => A_3,
    B_0 => B_0, B_1 => B_1, B_2 => B_2, B_3 => B_3,
    AbiggerB => AbiggerB, AequalB => AequalB, AsmallerB => AsmallerB);
    B_0 <= not B_0 after 10 ns;
    B_1 <= not B_1 after 20 ns;
    B_2 <= not B_2 after 40 ns;
    B_3 <= not B_3 after 80 ns;
    A_0 <= not A_0 after 160 ns;
    A_1 <= not A_1 after 320 ns;
    A_2 <= not A_2 after 640 ns;
    A_3 <= not A_3 after 1280 ns;
end Behavioral;

```

Here's the source that helped me achieve this:

<https://www.quora.com/Is-it-possible-to-toggle-a-signal-between-1-and-0-in-VHDL>

VHDL wave-function representation:



This design functions as it intended thanks to its modular design. Each step of the problem breaks down the problem into smaller and more clear and concise steps. It works as follows:

1. Separates the 4 bits of the given numbers in 2 pairs, where those 2 pairs are broken down in 2 more pairs as well.
2. Compares each bit with its counterpart.
3. At each step of division, a comparison result is generated, then this result is paired against the upper, more significant, results.
4. The result is the comparison of the 4-bit numbers.

Examples of it working as intended:

1000 > 0000

0001 < 0100

0100=0100

A_0	0
A_1	0
A_2	0
A_3	0
B_0	0
B_1	0
B_2	0
B_3	1
AbiggerB	0
AequalB	0
AsmallerB	1

A_0	0
A_1	0
A_2	1
A_3	0
B_0	1
B_1	0
B_2	0
B_3	0
AbiggerB	1
AequalB	0
AsmallerB	0

A_0	0
A_1	0
A_2	1
A_3	0
B_0	0
B_1	0
B_2	1
B_3	0
AbiggerB	0
AequalB	1
AsmallerB	0